ADR-004: 埋め込みモデル — Voyage AI voyage-3 を採用

ステータス

承認

日付

2026-03-12

コンテキスト

シャチョツーのナレッジエンジンは **ベクトル検索** を中核技術として使用する。ユーザーの質問に関連するナレッジを検索したり（RAG)、類似ナレッジの重複検出、セマンティックキャッシュ等、埋め込みモデルはプロダクト品質を直接左右する。

埋め込みが必要なユースケース

ユースケース	特性	精度要求
Q&Aエンジン（RAG検索）	ユーザーの質問と関連ナレッジチャンクのマッチング	最重要
ナレッジ重複検出	同一・類似のナレッジを自動統合	高
セマンティックキャッシュ	同一・類似クエリのLLMレスポンスを再利用	中
関連ルール発見	意思決定ルール間の意味的関連を検出	高
ゲノムマッチング	入力テキストと業界テンプレートの類似度計算	中

日本語品質の重要性

ターゲットユーザーは日本の中小企業。入力テキストには以下が含まれる:
- 建設業・製造業・歯科等の業界固有専門用語
- 口語体・方言混じりのビジネス会話（音声取り込み）

- 漢字・ひらがな・カタカナが混在する文書
- 略語・社内用語（例: 「ガイコウ」= 外交=営業担当者、等）

汎用英語モデルでは日本語の業界用語の意味を正しく捉えられず、Q&A精度が低下する。

ベクトル次元と性能のトレードオフ

次元数	検索品質	インデックスサイズ	クエリ速度
384次元	低〜中	小	速い
768次元	中	中	中
1024次元	高	中〜大	中
3072次元	最高	大	遅い

Supabase pgvectorのHNSWインデックスは1024次元での運用実績が豊富で、性能と品質のバランスが良い。

検討した選択肢

選択肢A: Voyage AI voyage-3 — 採用案

概要: Voyage AIが提供する多言語対応の埋め込みモデル。特に日本語・多言語テキストの検索品質で定評がある。

モデルスペック:
- 次元数: 1024
- 最大トークン長: 32,000 トークン
- 対応言語: 日本語を含む多言語

メリット:

1. **日本語品質トップクラス**: MTEB（Massive Text Embedding Benchmark）の日本語セクション・JMTEB（日本語MTEBベンチマーク）で一貫して高スコアを記録。OpenAI `text-embedding-3-large` と比較して日本語の意味検索でより正確な結果を返す
2. **長文対応**: 32,000トークンの入力長。業務マニュアル1章分を分割なしで埋め込める
3. **1024次元**: Supabase pgvectorのHNSWインデックスとの相性が良い。メモリ効率と検索品質のバランスが最適
4. **専門領域対応**: `voyage-3` の他に `voyage-finance-2`（金融）・`voyage-law-2`（法律）等のドメイン特化版が存在。将来的に業界特化モデルへの切り替えも容易

5. **コスト**: OpenAI `text-embedding-3-large` と同等〜やや安価。大量埋め込み時でも許容範囲
6. **APIシンプルさ**: RESTful API。`voyageai` Python SDKが整備されており実装が簡単

```python
import voyageai

client = voyageai.Client(api_key=settings.VOYAGE_API_KEY)
result = client.embed(
texts=["この建設現場の安全基準は..."],
model="voyage-3",
input_type="document" # or "query"
)
embeddings = result.embeddings # List[List[float]], 1024次元
```

7. **input_type 分離**: `document`（保存時）と `query`（検索時）を別々に最適化。asymmetric embeddingにより検索精度が向上

デメリット:
- **追加APIベンダー**: LLM（Gemini/Claude）とは別のAPIキー・利用規約・SLA管理が必要
- **障害リスク**: Voyage AI が障害の場合、全ての埋め込み処理がブロック
- 緩和策: フォールバックとしてOpenAI埋め込みAPIを設定
- **知名度**: Googleや OpenAI 比で知名度が低い。投資家・顧客への説明が必要な場面がある

選択肢B: OpenAI text-embedding-3-large — 却下

概要: OpenAIが提供する高品質な埋め込みモデル。3072次元（次元削減可能）。

メリット:
- **エコシステム**: LangChain等との統合が最も充実。サンプルコードが豊富
- **品質**: 英語・多言語でトップクラスの品質
- **知名度**: 「OpenAI製」という信頼性

デメリット:
- **日本語品質**: 英語に比べて日本語の意味検索精度が劣るというベンチマーク結果が複数報告されている。特に業界専門用語・口語体への対応が弱い
- **次元数**: 3072次元はSupabase pgvectorのHNSWインデックスメモリ使用量が大きくなる。次元削減（`dimensions` パラメータ）で1024次元に抑えることは可能だが、品質が低下する
- **コスト**: `text-embedding-3-large` は `text-embedding-3-small` の6倍のコスト。大量埋め込みではコスト差が積み上がる
- **OpenAI依存**: LLMでは第3選択肢としたOpenAIに、埋め込みでも依存することになり、リスク集中

→ **決定**: 日本語品質の劣位とOpenAI依存集中が問題。フォールバックとして構成するが、プライマリには採用しない。

選択肢C: Cohere embed-v3 — 却下

概要: Cohereが提供するビジネス用途向け埋め込みモデル。

メリット:
- **ビジネス文書**: ビジネス文書・長文ドキュメントの検索に最適化
- **多言語**: 100以上の言語に対応
- **input_type**: Voyage同様にdocument/queryの分離をサポート
- **コスト**: 競争力のある価格

デメリット:
- **日本語最適化**: Voyage AIに比べて日本語業界特化の最適化が弱い。JMTEBベンチマークでvoyage-3に劣る結果が多い
- **知名度（日本）**：日本市場での利用事例・コミュニティが少なく、トラブルシューティング情報が不足
- **統合実績**: LangChainとの統合はあるが、pgvectorとの組み合わせでの実績がvoyage AIより少ない

→ **決定**: 日本語品質でVoyage AIに劣る。第2フォールバックとして構成は可能だが、プライマリ採用しない。

選択肢D: Google text-embedding-004 (Vertex AI) — 却下

概要: GoogleがVertex AI上で提供する埋め込みモデル。Geminiと同一エコシステム。

メリット:
- **インフラ統合**: LLMをGeminiで採用しているため、認証・課金・セキュリティが一元化できる
- **品質**: 多言語で高品質

デメリット:
- **Googleへの集中依存**: LLM（Gemini）・インフラ（Cloud Run）に加えて埋め込みもGoogleに依存すると、Googleの価格改定・規約変更・障害の影響が最大化する。リスク分散の観点から集中は避けるべき
- **日本語特化**: Voyage AIと比較して日本語の業界特化ベンチマークが少なく、優位性が不明確
- **ベンダーロックイン**: Googleへのフルスタック依存は、競合他社（Microsoft・Amazon等）のSaaS企業を顧客に迎える際の懸念材料になり得る

→ **決定**: ベンダー集中リスクが最大の課題。採用しない。

選択肢E: ローカル埋め込みモデル (intfloat/multilingual-e5, cl-nagoya/sup-simcse-ja) — 却下

概要: Hugging Face等から日本語対応の埋め込みモデルをダウンロードし、Cloud Runで実行。

メリット:
- **コスト**: APIコールなし。変動費ゼロ
- **レイテンシ**: ローカル推論でAPIレイテンシなし
- **データ**: 外部APIにテキストを送信しない

デメリット:
- **品質**: `multilingual-e5-large`（768次元）はVoyage AIより検索精度が低い。Q&A精度の低下がプロダクト価値に直結
- **インフラ**: GPU（CPU実行では低速）を搭載したCloud Runインスタンスが必要。コスト・管理コストが上昇
- **スケーリング**: 高負荷時にCPU/GPU不足が発生。API型はプロバイダーが自動スケール
- **モデル更新**: 新しいモデルへの更新が手動作業になる

→ **決定**: 品質とインフラコストの問題で却下。将来的にデータ主権要件が生じた際に再評価。

決定

Voyage AI `voyage-3` をプライマリ埋め込みモデルとして採用する。

フォールバック順:

```
1st: voyage-3 (Voyage AI)          ← 日本語品質最優先
2nd: text-embedding-3-large (OpenAI) ← voyage-3 障害時
3rd: embed-v3 (Cohere)             ← 緊急時のみ
```

実装方針

埋め込み処理は `brain/knowledge/embedder.py` に集約し、プロバイダー切り替えを抽象化する:

```
class EmbeddingClient:
    """埋め込みモデルの抽象化レイヤー"""

    async def embed_documents(
        self,
        texts: list[str],
        model_tier: EmbeddingTier = EmbeddingTier.PRIMARY,
    ) -> list[list[float]]:
        """文書埋め込み（保存時用）"""
        ...

    async def embed_query(
        self,
        text: str,
        model_tier: EmbeddingTier = EmbeddingTier.PRIMARY,
    ) -> list[float]:
        """クエリ埋め込み（検索時用）"""
        ...
```

pgvector インデックス設定

```
-- knowledge_items テーブルの埋め込みカラム
ALTER TABLE knowledge_items
  ADD COLUMN embedding vector(1024);

-- HNSWインデックス（コサイン類似度）
CREATE INDEX ON knowledge_items
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- 検索時のクエリ（RLS + ベクトル検索の組み合わせ）
SELECT id, content, 1 - (embedding <=> $1) AS similarity
FROM knowledge_items
WHERE company_id = $2   -- RLSフィルタ（必須）
ORDER BY embedding <=> $1
LIMIT 10;
```

モデル変更時の再埋め込み手順

埋め込みモデルを変更した場合（例: voyage-3 → voyage-3.5）、既存ベクトルとの次元・スペース互換性がない。以下の手順が必要:

1. 新モデル用の埋め込みカラム追加（`embedding_v2 vector(1024)`）
2. バックグラウンドジョブで全レコードを再埋め込み
3. 新カラムで検索 → 品質確認
4. 旧カラム削除

この再埋め込みコストを考慮して、モデル変更は慎重に行う。

影響

ポジティブな影響

- **Q&A精度向上**: 日本語業界用語への対応により、関連ナレッジの検索精度が向上。Q&A回答品質が直接改善
- **重複検出精度**: 類似ナレッジの自動統合精度が上がり、ナレッジベースの品質維持コストが下がる
- **セマンティックキャッシュ効果**: 類似クエリのキャッシュヒット率が上がり、LLMコストを削減

負の影響・トレードオフ

- **追加ベンダー管理**: Voyage AIのAPIキー・料金プラン・SLA管理が必要。年間サービス終了リスクも存在（ただしAPIの後継モデルへの移行パスは通常提供される）
- 緩和策: 埋め込み抽象化レイヤー（`EmbeddingClient`）によりプロバイダー切り替えは1日以内で可能
- **モデル変更コスト**: voyage-3 から別モデルへの切り替え時、全ナレッジの再埋め込みが必要。100万件のナレッジで数時間〜数日のバックグラウンド処理が発生

今後の制約

1. **埋め込み次元の固定**: `knowledge_items.embedding` は1024次元に固定。モデル変更時は移行手順に従う
2. **全埋め込み処理は `EmbeddingClient` 経由**: 直接 `voyageai` ライブラリを呼び出すことを禁止
3. **input_type の使い分け**: 保存時は `document`、検索クエリ時は `query` を必ず指定する
4. **バッチ処理**: 大量テキストの埋め込みはバッチAPIを使用してAPI呼び出し回数を最小化（voyage-3の最大バッチサイズ: 128件）

関連

- `shachotwo/02_プロダクト設計.md` — Section 5: DB設計（knowledge_items.embedding）
- `brain/knowledge/embedder.py` — 埋め込みクライアント実装
- ADR-001: Gemini LLM採用（LLMとは異なる役割。埋め込みはvoyage、生成はGemini）
- ADR-002: Supabase採用（pgvectorでの格納先）